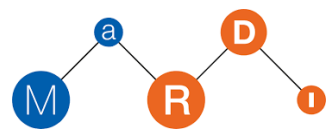
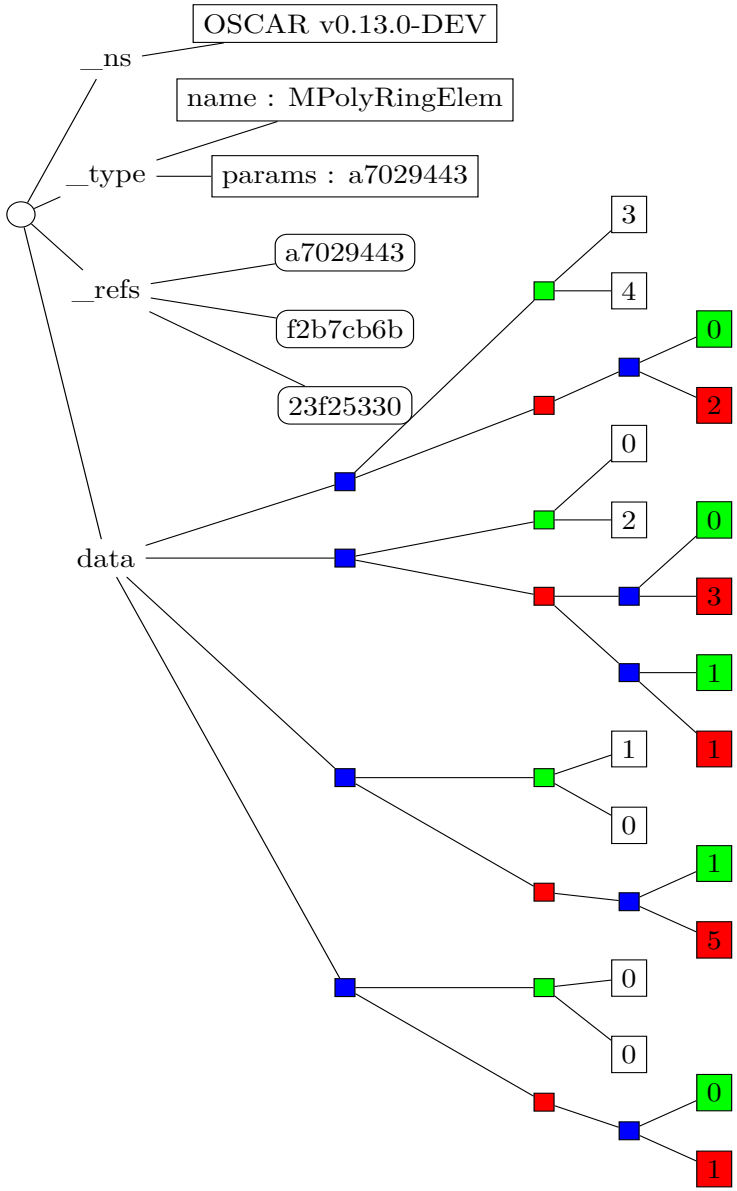
The `mrdi` File Format

- JSON file format.
- Similar to `polymake` format but generalizes to include algebraic data.
- Namespaces for semantic seperation.
- References stored with UUIDs handle the isomorphism problem.



Tree Structure



$2y^3z^4$

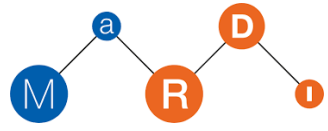
$(a+3)z^2$

$5ay$

1

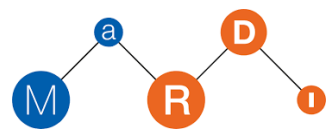
Example File Serialized with OSCAR


```
{
  "_ns": {"Oscar": ["https://github.com/oscar-system/Oscar.jl",
                    "1.7.0-DEV-d655de93da67af8036060334e848cf151b84683d"] },
  "_type": {  "name": "MPolyRingElem",
              "params": "869a359a-43d3-43f4-9821-0af9346be019"  },
  "data": [[[ "3", "4"], [[ "0", "2"]]],
           [[ "1", "0"], [[ "1", "5"]]],
           [[ "0", "2"], [[ "0", "3"], [ "1", "1"]]],
           [[ "0", "0"], [[ "0", "1"]]]]

  "_refs": {
    "152ac7bd-e85a-4b36-acc2-743ade2cad4f": {
      "_type": { "name": "PolyRing",
                 "params": "9cfbeeeb-e345-4c8d-a074-e8466bdfeb9a" },
      "data": { "symbols": [ "x" ] }
    },
    "869a359a-43d3-43f4-9821-0af9346be019": {
      "_type": { "name": "MPolyRing",
                 "params": "a8309b96-caec-443c-bedb-e23bb0634c14" },
      "data": { "symbols": ["y", "z" ] }
    },
    "9cfbeeeb-e345-4c8d-a074-e8466bdfeb9a": {
      "_type": { "_instance": "FqField",
                 "name": "FiniteField"  },
      "data": "7"
    },
    "a8309b96-caec-443c-bedb-e23bb0634c14": {
      "_type": { "_instance": "FqField",
                 "name": "FiniteField",
                 "params": "152ac7bd-e85a-4b36-acc2-743ade2cad4f" },
      "data": [ [   "0",   "1"   ],
                [   "2",   "1"   ]]
    }
  },
}
```

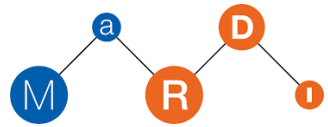
Implementation Details

- Two main tree branches `_type` and `data`.
- Type registration, used for security, type stringification and declaring which types need ids (refs).
- `type_params` function does a shallow pass, gathering type and parameter information.
- `save_typed_object` / `load_typed_object`, high level functions.
- `save_type_params` / `load_type_params`, used for serializaing the type branch. (overloaded for container types)
- `save_object` / `load_object`, low level, each type has their own method.
- Global Serializer state for persitences across many files.



Finite Field Serialization Implementation

```
@register_serialization_type FqField "FiniteField" uses_id
@register_serialization_type FqFieldElem

function type_params(K::FqField)
  absolute_degree(K) == 1 && return TypeParams(FqField, nothing)
  return TypeParams(FqField, parent(defining_polynomial(K)))
end

function save_object(s::SerializerState, K::FqField)
  if absolute_degree(K) == 1
    save_object(s, order(K))
  else
    save_object(s, defining_polynomial(K))
  end
end

function load_object(s::DeserializerState, ::Type{<: FqField}, params::PolyRing)
  finite_field(load_object(s, PolyRingElem, params), cached=false)[1]
end

function load_object(s::DeserializerState, ::Type{<: FqField})
  finite_field(load_object(s, ZZRingElem, ZZRing()))[1]
end

# elements
function save_object(s::SerializerState, k::FqFieldElem)
  K = parent(k)

  if absolute_degree(K) == 1
    save_object(s, lift(ZZ, k))
  else
    poly_parent = parent(defining_polynomial(K))
    polynomial = lift(poly_parent, k)
    save_object(s, polynomial)
  end
end

function load_object(s::DeserializerState, ::Type{<: FqFieldElem}, K::FqField)
  load_node(s) do _
    if absolute_degree(K) != 1
      return K(load_object(s, PolyRingElem, parent(defining_polynomial(K))))
    end
    K(load_object(s, ZZRingElem, ZZRing()))
  end
end
```



Rosetta Stone Prototype

https://oscar-system.github.io/rosetta-stone-db_prototype/